

03/10/2023

Sistema de Administración de Turnos para Clínica odontológica

Plan de Desarrollo Software

V0.4



HISTORIAL DE REVISIONES

Fecha	Versión	Descripción	Autor
13/06/2023	0.1	Versión preliminar como propuesta de desarrollo.	Ceballos Palacios, Santiago Vargas Lanzone, Adrian
18/07/2023	0.2	Versión Etapa de desarrollo.	Ceballos Palacios, Santiago Vargas Lanzone, Adrian
12/08/2023	0.3	Revisión de calidad	Ceballos Palacios, Santiago Vargas Lanzone, Adrian
15/09/2023	0.4	Revisión de calidad	Ceballos Palacios, Santiago Vargas Lanzone, Adrian

CONTROL DE CAMBIOS (doc)

Revisión	Descripción del cambio	Fecha de vigencia
0.0	Creación del documento.	06/06/2023
0.1	Se corrigieron algunos problemas de nomenclatura en ciertas partes del documento. Se agregaron diagramas de “Caso de uso”.	10/06/2023
0.2	Se incorporo nueva información para la “Etapa de Desarrollo”, explicando en detalle la implementación del programa. Se corrigió y agrego información extra a la “Vista general del Proyecto”. Se agregaron diagramas de “Diagramas de Clases”. Se incorporo índice en el “Panel de Navegación”, listo y preparado para el estándar PDF/a.	18/07/2023
0.3	Se corrigieron errores de gramática. Se corrigieron los diagramas presentados en el documento. Se añadió información extra sobre contenido relacionado a calidad/seguridad.	12/08/2023
0.4	Se corrigió información referida al proyecto general. Se modificaron los Requisitos funcionales y no funcionales. Se modificaron los Diagramas UML y Modelos de Caso de Uso. Se agrego información extra a los Requisitos de Interfaz (Usuario y Hardware) Se modificaron las fases del proyecto, delimitadas correctamente.	15/09/2023



Índice

1. Introducción	4
1.1 Propósito	4
1.2 Alcance	4
1.3 Resumen	4
2. Vista General del Proyecto	5
2.1 Propósito, Alcance y Objetivos	5
2.2 Vision	5
2.3 Requisitos Funcionales	6
2.4 Requisitos no Funcionales	7
2.5 Modelo de Objetos del Negocio (Caso de uso)	8
2.6 Diagrama Caso de uso (General)	9
2.7 Especificación para Caso de uso	10
2.8 Diagrama E-R (Entidad-Relación)	11
3. Requisito de Interfaz	12
3.1 Interfaces de Usuario	12
3.2 Interfaces de hardware	12
3.3 Interfaces de software (requisitos)	12
3.4 Interfaces de comunicación	13
3.5 Requisitos de seguridad	13
3.6 Calidad del software	13
4. Gestión de Proyecto	14
4.1 Estimación del proyecto	14
4.2 Fases del proyecto	14
4.3 Descripción de las fases	15
4.4 Modelo MVC (Modelo-Vista-Controlador)	16
4.5 Diagrama del Modelo MVC	17
5. Glosario	18



Sistema de Administración de Turnos para Clínica odontológica	Versión: 0.4
Catedra: Programación II	
Consultora: PROMPT Engineering	

Plan de Desarrollo del Software

1. Introducción

En este documento, se establece el plan de desarrollo de software para la creación de un sistema de administración de una clínica odontológica. Nuestro objetivo es brindar una solución eficiente y centralizada que permita gestionar los datos de los pacientes, asignar turnos a los odontólogos.

4.5 Propósito

El propósito de este plan es proporcionar una guía clara y detallada para el diseño, implementación y puesta en marcha del sistema de administración de la clínica odontológica. Buscamos mejorar la eficiencia y precisión en la gestión de los datos de los pacientes, así como facilitar el trabajo de los odontólogos y secretarios, optimizando los procesos y ofreciendo una atención de calidad.

1.2 Alcance

Nuestro proyecto se enfoca en el desarrollo e implementación de una aplicación web para la administración de datos y turnos en una clínica odontológica. El sistema permitirá el registro de pacientes, incluyendo información personal.

1.3 Resumen

El sistema de administración de la clínica odontológica es una aplicación web diseñada para optimizar la gestión de datos y turnos de pacientes. Su objetivo es mejorar la eficiencia y brindar atención de calidad a través de funcionalidades clave. Estas incluyen el registro completo de pacientes, con información personal. En definitiva, se espera que este sistema optimice la gestión de la clínica y facilite la toma de decisiones basadas en datos precisos y actualizados.



2. Vista General del Proyecto

2.1 Propósito, Alcance y Objetivos

El proyecto tiene como propósito principal desarrollar un sistema de administración para una clínica odontológica a través de una aplicación web. El alcance del proyecto abarca diversas funcionalidades, como el registro de pacientes, la asignación de turnos y la posibilidad de realizar consultas.

El objetivo fundamental de este sistema es mejorar la eficiencia en la gestión de datos de los pacientes. Para lograrlo, se buscará optimizar los procesos internos de la clínica, permitiendo una gestión más ágil y eficiente de los registros y los turnos.

2.2 Visión

Nuestra visión en este proyecto consiste en crear una aplicación web para administrar una clínica odontológica. El objetivo es permitir a los usuarios (odontólogos y secretarios) registrar y mantener los datos de los pacientes, programar y administrar los turnos con los diferentes odontólogos.

Esta solución digital tiene como propósito principal mejorar la eficiencia operativa, optimizar la gestión de la información de los pacientes y facilitar una programación precisa y sencilla de los turnos.

A través de esta aplicación, buscamos elevar la calidad de la atención al paciente y aumentar la eficiencia del personal de la clínica. El sistema estará diseñado con un enfoque en la usabilidad y accesibilidad, de manera que todos los usuarios, desde los secretarios hasta los odontólogos, puedan utilizar la plataforma de forma efectiva.

En definitiva, esta aplicación se convertirá en una pieza central para la gestión de la clínica, desempeñando un papel crucial en la prestación de servicios de atención odontológica de alta calidad para todos los pacientes.



Sistema de Administración de Turnos para Clínica odontológica	Versión: 0.4
Catedra: Programación II	
Consultora: PROMPT Engineering	

2.3 Requisitos Funcionales

Registro de pacientes:

- a) Captura de datos personales del paciente (nombre, edad, dirección, etc.).
- b) Gestión de odontólogos.
- c) Registro de odontólogos con sus respectivas especialidades.
- d) Asignación de odontólogos a los turnos de los pacientes.

Control de acceso:

- a) Sistema de login.

Sistema de turnos:

- a) Pacientes asignados a un odontólogo por turno.



2.4 Requisitos No Funcionales

Usabilidad:

- a) Interfaz de usuario intuitiva y fácil de usar.
- b) Tiempo de respuesta rápido para mejorar la eficiencia en el uso del sistema.

Seguridad:

- a) Control de acceso seguro con autenticación.
- b) Protección de datos personales y médicos de los pacientes.

Escalabilidad y rendimiento:

- a) Capacidad para manejar una gran cantidad de registros de pacientes y turnos sin degradar el rendimiento.
- b) Escalabilidad para adaptarse al crecimiento futuro de la clínica.

Portabilidad:

- a) Compatibilidad con diferentes navegadores web y dispositivos.
- b) Posibilidad de acceder al sistema desde diferentes ubicaciones o dispositivos.

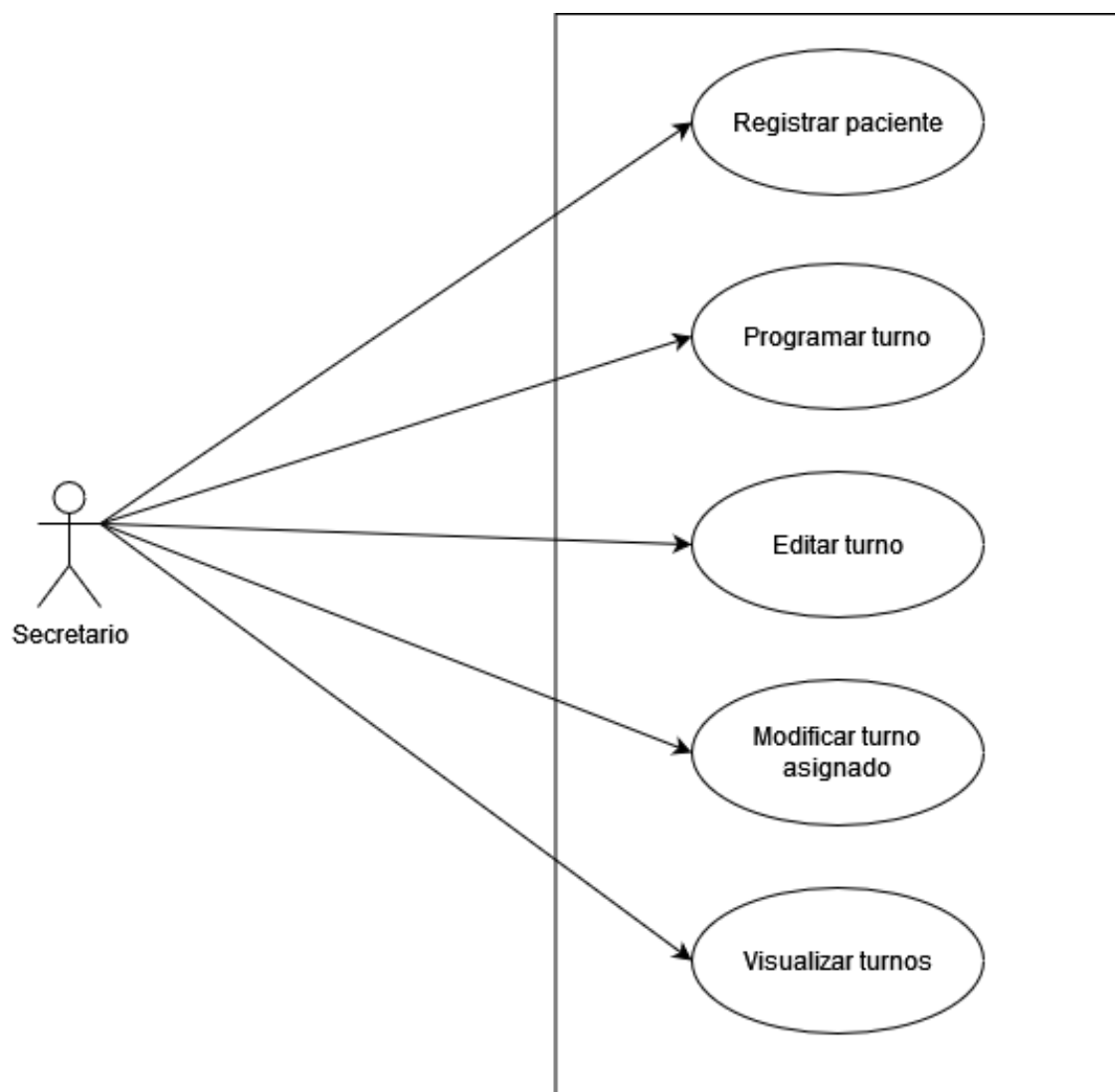
Mantenibilidad:

- a) Código limpio y bien estructurado para facilitar futuras modificaciones y mejoras.
- b) Documentación completa y clara para ayudar en el mantenimiento del sistema.
- c) Uso de buenas prácticas de desarrollo de software para garantizar la calidad del código.



2.5 Modelo de Objetos del Negocio

CASO DE USO

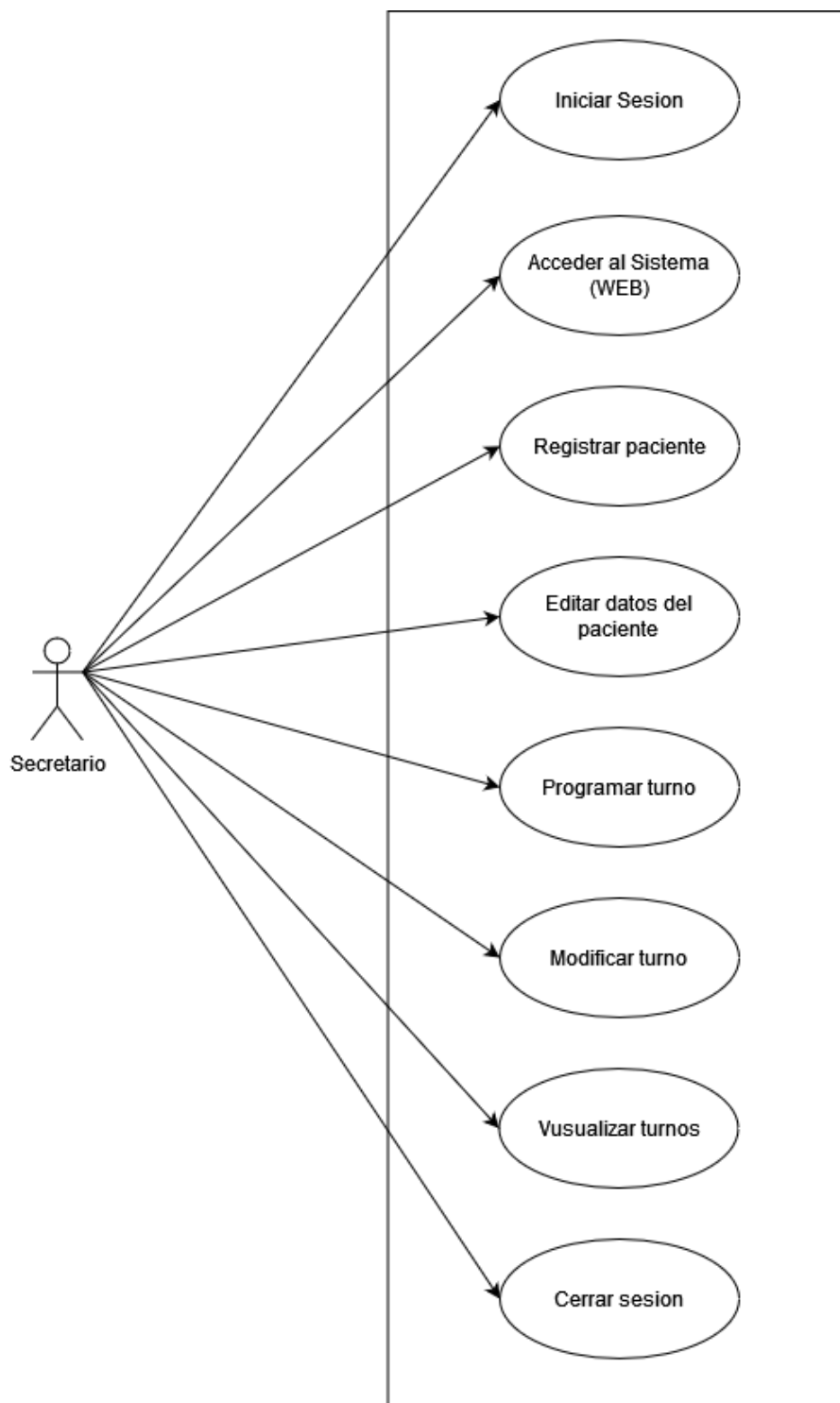


Realizado en [Draw.io](https://draw.io)



2.6 Diagrama de Caso de Uso (General)

SISTEMA DE ADM DE TURNOS



Realizado en [Draw.io](https://draw.io)



Sistema de Administración de Turnos para Clínica odontológica	Versión: 0.4
Catedra: Programación II	
Consultora: PROMPT Engineering	

2.7 Especificación para Caso de Uso

Identificación del Caso de Uso: registrar Nuevo Paciente

Actor(es): secretario/a

Precondiciones: el secretario ha iniciado sesión en el sistema.

Flujo Principal:

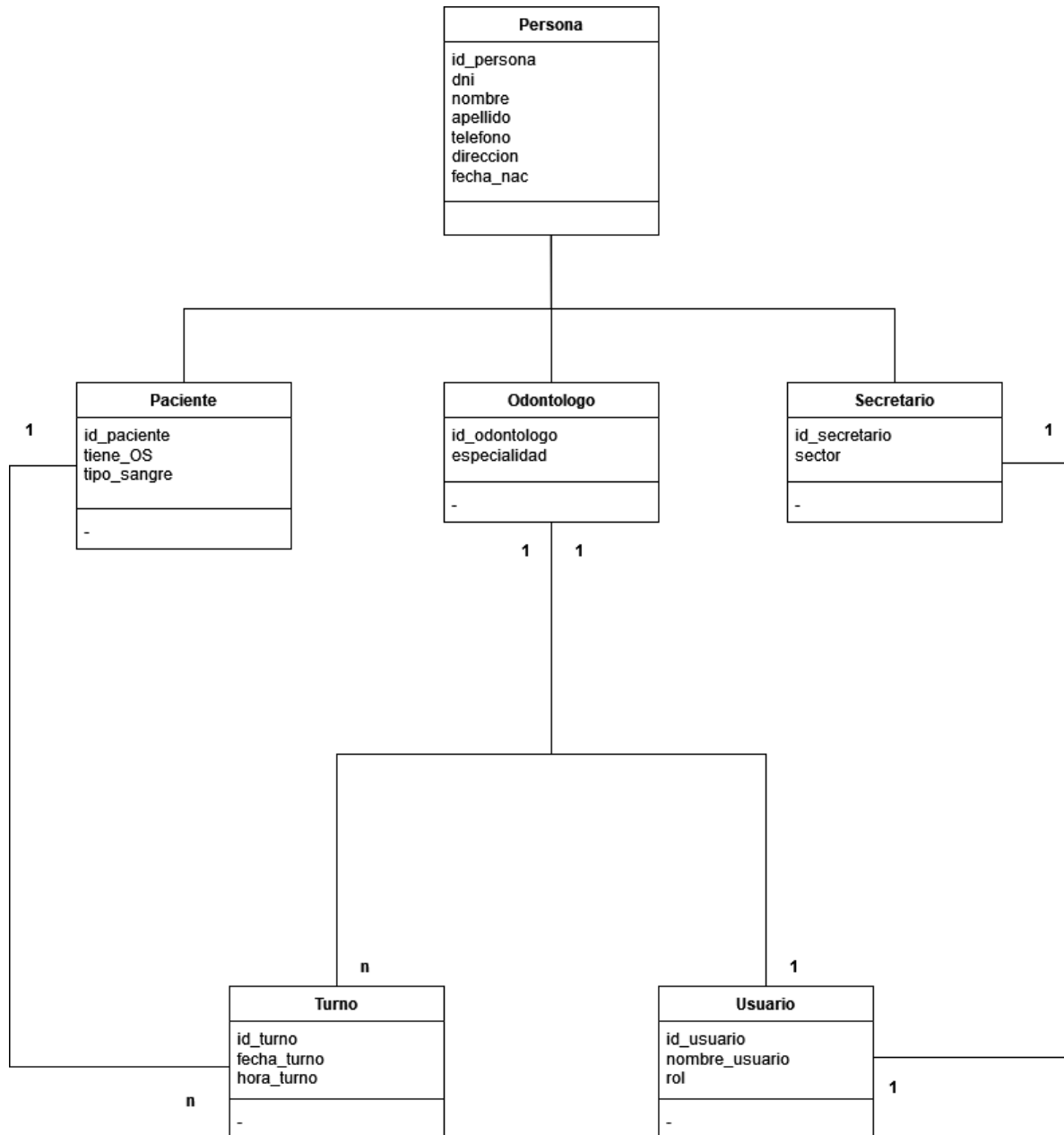
- 1) El secretario selecciona la opción “Alta paciente “un nuevo paciente en la aplicación.
- 2) El sistema presenta un formulario para ingresar los datos del nuevo paciente.
- 3) El secretario ingresa la información necesaria del paciente, que incluye nombre, apellido, dirección, número de teléfono.
- 4) El secretario selecciona la opción para guardar la información.
- 5) El sistema valida la información ingresada.
- 6) Si la información es válida, el sistema registra al nuevo paciente.

Flujos Alternativos:

Si la información ingresada es inválida (por ejemplo, campos obligatorios en blanco o información incorrecta), el sistema muestra un mensaje de error y solicita al secretario que corrija los errores.



2.8 Diagrama de Clases E-R (Entidad-Relación)



Realizado en [Draw.io](https://draw.io)



3. Requisitos de la Interfaz

3.1 Interfaces de usuario

- Software front-end: HTML – JSP, CSS.
- Software de fondo: SQL, Java.

3.2 Interfaces de hardware

- Pestañas
- Un navegador WEB, compatible con HTML, CSS, JSP y Java.

3.3 Interfaces de software

A continuación, mostramos el software que será implementado para la aplicación que estamos desarrollando, estos requisitos tienen el objetivo de aclarar de forma concreta que es lo que utilizaremos.

Software utilizado	Descripción
Sistema operativo	Windows por su mejor soporte y facilidad de uso para el cliente.
Base de datos	Para guardar los datos necesarios y requeridos para nuestro sistema, hemos elegido la base de datos SQL.
Java	Para implementar el proyecto hemos elegido el lenguaje Java, por contar con un gran soporte y facilidad de manejo de código.
HTML-JSP	En el apartador visual, utilizaremos HTM-JSP y CSS, de esa forma daremos un aspecto grafico agradable y bien diseñado.



3.4 Interfaces de comunicación

Este proyecto es compatible con todos los tipos de navegadores web, hemo realizado un pequeño relevamiento interno y pudimos verificar que cada navegador en la actualidad será capaz de ejecutar nuestro proyecto, de los cuales “**Firefox**” y “**Google Chrome**” son los más recomendados y los que tuvieron un mejor desempeño.

3.5 Requisitos de seguridad

Si se produce un daño en alguna parte de la base de datos, debido a una falla de software o de hardware, como podría ser el fallo de un disco, es de vital importancia realizar copias de seguridad a la base de datos, con la intención de restaurarla posteriormente. Nuestro método realiza copias de seguridad regularmente para evitar perdida de datos y así reconstruir la base nuevamente a un estado actualizado.

3.6 Calidad del software

- ❖ DISPONIBILIDAD: El sistema de administración de la clínica debe estar disponible para realizar citas y gestionar los datos de los pacientes en cualquier momento, ya que los pacientes pueden necesitar agendar citas en diferentes horarios del día.
- ❖ EXACTITUD: El sistema debe registrar con precisión las fechas, horarios y detalles de cada cita, asegurando que los pacientes sean asignados a los odontólogos correctos y que se registren los tratamientos adecuados.
- ❖ MANTENIMIENTO: El personal calificado debe poder realizar el mantenimiento y actualizaciones del sistema sin afectar su funcionalidad, garantizando que esté siempre disponible para su uso.
- ❖ USABILIDAD: El sistema de administración de la clínica debe ser intuitivo y fácil de usar para que el personal de la clínica pueda realizar las tareas de agendamiento, gestión de pacientes y generación de informes de manera eficiente y sin dificultades.



4. Gestión del Proyecto

4.1 Estimación del proyecto:

PROMPT Engineering: la consultora tiene como objetivo finalizar este proyecto los últimos días de Julio/mediados de Agosto como fechas de desarrollo máximas. Son fechas ajustadas, pero nos permitirá contar con un margen de tiempo suficiente para dar lugar a los plazos de exposiciones necesarios, y empezar a trabajar en la presentación de nuestro programa, así también diseñar un “**Manual de Usuario**”, el cual también está dentro de nuestro objetivo.

4.2 Fases del Proyecto

Por una cuestión de orden, decidimos separar el Proyecto en “Fases”, permitiendo tener un mejor orden en cada una de las fases, y así ir de acuerdo con nuestros tiempos, de esa forma podremos presentar nuestro Proyecto en tiempo y forma.

Fase	Nro. Iteraciones	Duración
Fase de Inicio	1	1 semanas
Fase de Desarrollo	2	3 semanas
Fase de Verificación y Pruebas	-	1.5 semanas



4.3 Descripción de las Fases

Aquí se presenta de manera detallada cada una de las fases y como decidimos componer cada fase en relación con el proyecto y nuestro tiempo:

Descripción	Hito
Fase de Inicio	En esta fase se desarrollarán los primeros detalles del proyecto, dejando en claro el “Propósito”, definiendo “Alcances” y “Objetivos”. Presentando una vista general del proyecto, y nuestro Plan de desarrollo, cubierto con toda la información necesaria para dejar en claro nuestro objetivo, además de Diagramas que muestran de forma gráfica nuestro sistema.
Fase de Desarrollo	En esta fase se analiza todos los requisitos anteriores y se comienza con el desarrollo de un prototipo. Esta fase está compuesta con muchas pruebas. La revisión de nuestro prototipo en esta etapa se somete a revisiones de muy corta duración y si el prototipo es estable y cumple con nuestras demandas, se comienza con una nueva rama donde el desarrollo es integro y completo, dando lugar a la primera iteración estable. Esta fase de desarrollo tiene una duración aproximada de 3 semanas.
Fase de Verificación y Pruebas	En esta fase se prepara nuestro programa y se somete a prueba en diferentes entornos de escritorio, asegurando la estabilidad y el funcionamiento del mismo. A esta altura, nuestro programa ya se considera funcionar y esta sobre una rama “Principal”, que es donde la consultora está prácticamente preparada para el despliegue y presentación del software.



4.5 Modelo MVC

Modelo-Visión-Controlador (MVC) es un patrón de diseño de software que divide la lógica de una aplicación en tres componentes interconectados: Modelo, Visión y Controlador. Este patrón de diseño está orientado principalmente a aplicaciones con una interfaz de usuario.

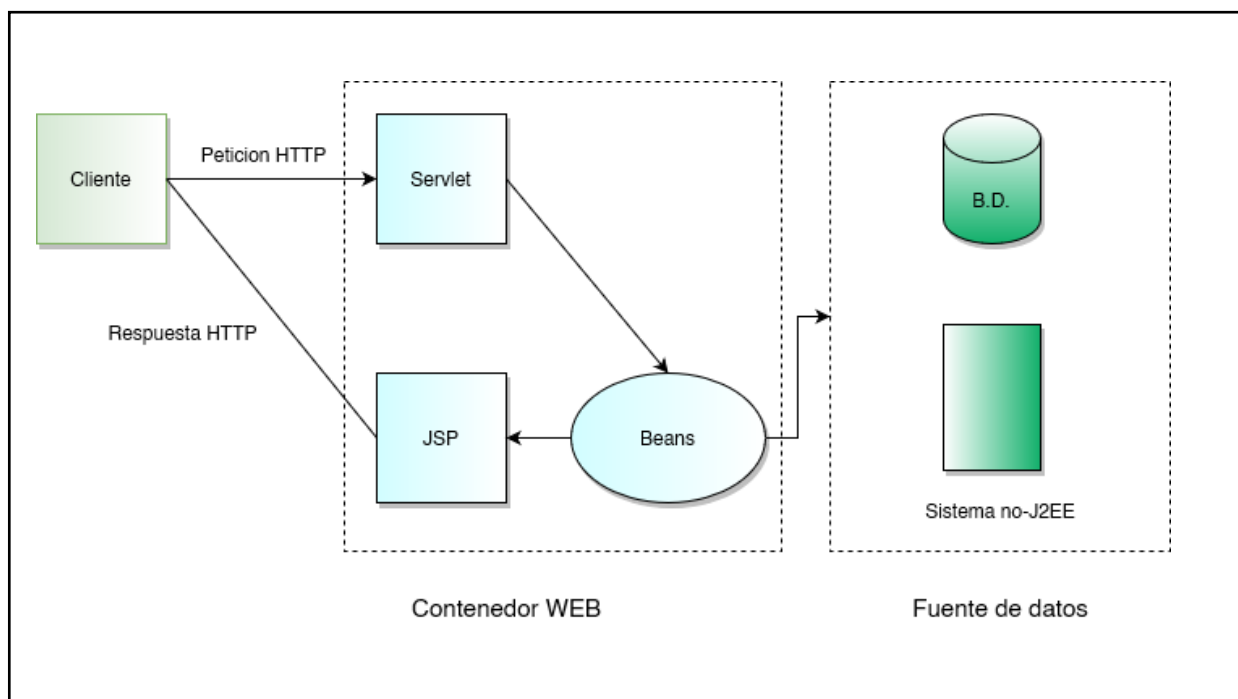
- **Modelo (Model):** Es la representación de los datos y la lógica de negocio. Se encarga de acceder a la base de datos, consultar datos, guardar y actualizarlos. Es decir, maneja todo lo relacionado con la información y las operaciones que se pueden hacer con ella.
- **Visión (View):** Es la representación gráfica de la información, es decir, lo que el usuario ve y con lo que interactúa directamente. Corresponde a la interfaz de usuario, que puede ser cualquier salida de representación de la información, como una página web, una aplicación móvil o un escritorio.
- **Controlador (Controller):** Actúa como intermediario entre el Modelo y la Visión. Se encarga de recibir las acciones del usuario desde la Visión, procesarlas (con la posible actualización del Modelo) y devolver una respuesta que se mostrará en la Visión. Es decir, controla y gestiona las interacciones del usuario.

El principal beneficio de usar el patrón MVC es la separación de preocupaciones, lo que significa que la lógica de negocio, la interfaz de usuario y las acciones del usuario están separadas en diferentes secciones del código. Esto facilita la mantenibilidad, la escalabilidad y la posibilidad de trabajar en paralelo en diferentes componentes.



Sistema de Administración de Turnos para Clínica odontológica	Versión: 0.4
Catedra: Programación II	
Consultora: PROMPT Engineering	

4.5 Diagrama simple del “Modelo MVC”



Realizado en [Draw.io](https://draw.io)



5. Glosario

Requisitos Funcionales: Estos son los detalles que describen qué debe hacer nuestro programa. Son las funciones y características específicas que queremos que tenga.

Requisitos no Funcionales: Son puntos importantes que nuestro programa debe cumplir, pero que no necesariamente están directamente relacionadas con lo que hace.

Modelo de Objetos del Negocio (Caso de uso): En esencia es como un mapa que muestra cómo diferentes puntos específicos en nuestro sistema se comunican entre sí. Esto ayuda a entender cómo los usuarios interactúan con nuestro programa y cómo funciona todo en conjunto.

Diagrama E-R (Entidad-Relación): Es un diagrama gráfico con relaciones que nos ayuda a ver qué cosas son importantes para nuestro sistema y cómo están conectadas entre sí. Es un esquema que contiene información.

Interfaces de software (requisitos): Estas son las reglas y especificaciones que definen cómo diferentes componentes del software se comunican entre sí. Cuando desarrollamos un sistema complejo, los diferentes módulos y partes deben trabajar juntos de manera armoniosa. Las interfaces de software aseguran que todos los elementos del sistema se entiendan y cooperen adecuadamente.

Interfaces de hardware: Básicamente son las conexiones de uso medio que existen entre software y hardware. El navegador Web podría ser uno de ellos, ya que hace uso de la interfaz de red usando el puerto RJ45.

Especificación para Caso de uso: Es un documento que nos cuenta cómo se supone que nuestro programa debe actuar en diferentes situaciones. Es como una guía de cómo los usuarios interactuarán con nuestro sistema.

Requisitos de seguridad: Estos son los detalles que debemos tener en cuenta para que nuestro sistema sea seguro y nadie pueda acceder a cosas que no deberían. Queremos proteger la información y asegurarnos de que todo funcione sin problemas.

Calidad del software: Aquí hablamos de qué tan bueno es nuestro programa en general. Queremos que sea rápido y que responda bien a lo que los usuarios necesitan.